INTERNAL HANDBOOK ARTISYN · HARNESS

The Artisyn Harness

What it is, why teams find it useful, and how it actually runs a project end to end.

- Actively used in delivery
- Root context `AGENTS.md` — cross-tool
- Package `v0.3.0`

PART ONE — WHAT IT IS

01 What it is, and what it's for

The Artisyn Harness is a structured workflow system layered on an AI coding assistant. Instead of prompting the assistant ad-hoc, work runs through **specialised, lifecycle-aware agents** governed by encoded conventions (skills) and feedback loops (reflections → skill library).

Its purpose is three things: **repeatability** — the same lifecycle steps run every ticket, quality doesn't depend on remembering the right prompts; **convergence** — conventions discovered on one ticket become skills inherited by the next; and **architecture coherence** — planning is audited against target architecture before any code is written.

The harness is portable by design: `docs/Harness/package/` is a self-contained package that instantiates it in any project — copy the folder, run the bootstrap recipe. It also proves itself on real work, not just in theory: its own canonical source is built entirely through it, every commit produced by `/design`, `/dev`, or `/skill-*`.

Runs on

`AGENTS.md` as the canonical root context file — skills and agents map onto Claude Code, Codex CLI, Cursor, Kiro, and Windsurf alike.

Bootstrapped by

Copy `docs/Harness/package/` into a new repo, then point an AI assistant at its `01 Bootstrap.md`.

Core principle

"Reduce scaffolding over time" — every ticket faster, as discovered conventions become skills inherited by the next.

PART TWO — WHY IT MAKES SENSE

02 Where it proves out

The same harness carries a project from before it's even sold to active delivery — not three separate toolchains stitched together.

👷 Pre-Sales
Estimation, initial HLD sketch, and client alignment before a contract exists.

→

🔍 Discovery
Bootstrap the harness onto the project, draft the HLD, shape context from what's actually there.

→

⚙️ Development
`/design` and `/dev` cycles on real tickets — the harness is now the delivery mechanism.

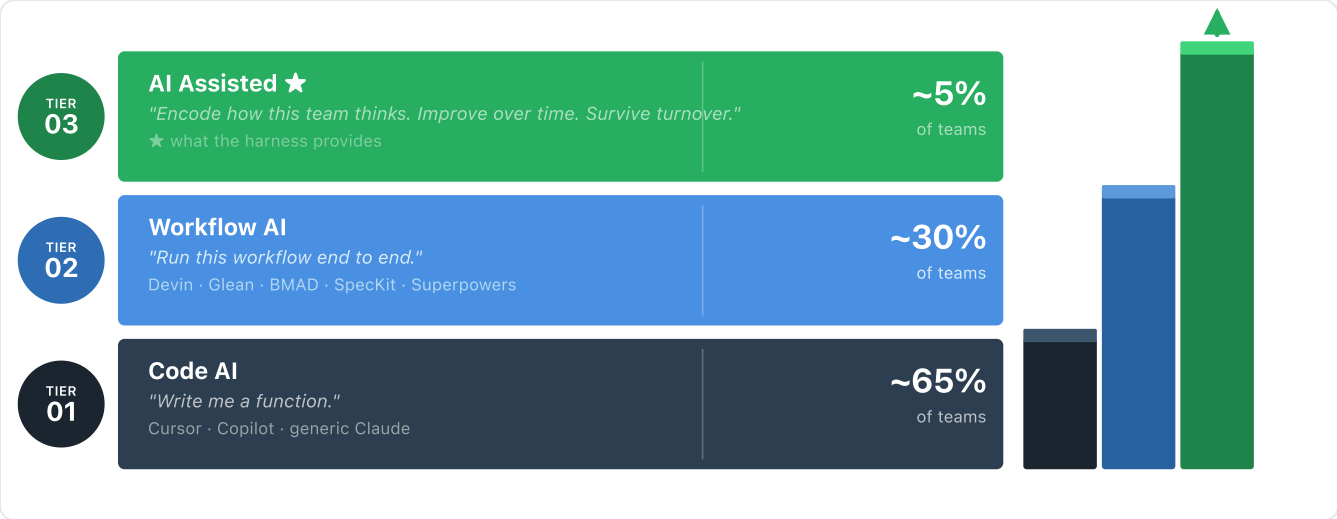
Full detail per phase — including what continues after delivery starts (Operate, Improve) — lives at `docs/Harness/04 Adoption/`.

MANAGES PROJECT CONTEXT — OR PRODUCT CONTEXT ACROSS PROJECTS

Most deployments run Project-scoped: one repo, one target architecture, reviewed by `@architect`. The package also ships a `product-architect` agent template for the other shape — a shared governance layer that several downstream project repos are accountable to, reasoning about whether a design honors product-level principles without assuming any one repo's internals. Same harness, either scope.

03 Why teams find it useful

Most AI coding tools operate at one of two lower tiers — useful, but they forget everything the moment the session ends. The harness is built for the tier above: it **encodes how a team thinks**, so that knowledge improves over time and survives turnover instead of living only in whoever's currently on the project.



Indicative distribution of where teams sit today, not measured data — the point is the ordering, not the precision.

WHAT "ENCODES HOW THE TEAM THINKS" ACTUALLY MEANS

Not a slogan — specific, machine-followable rules a team already lives by, written down once instead of re-explained every session: **which doc a decision belongs in** (the HLD/LLD placement matrix in `dev-document/SKILL.md`), **who's allowed to touch what** (agent tool-scoping — `@web` can't edit `apps/infra/`), and **which operations need a human** (guarded commands — `cdk deploy`, `git push`). Encoded once, applied identically on ticket 1 and ticket 400.

CODE BECOMES A DERIVATION, NOT THE SOURCE — A WORKED EXAMPLE

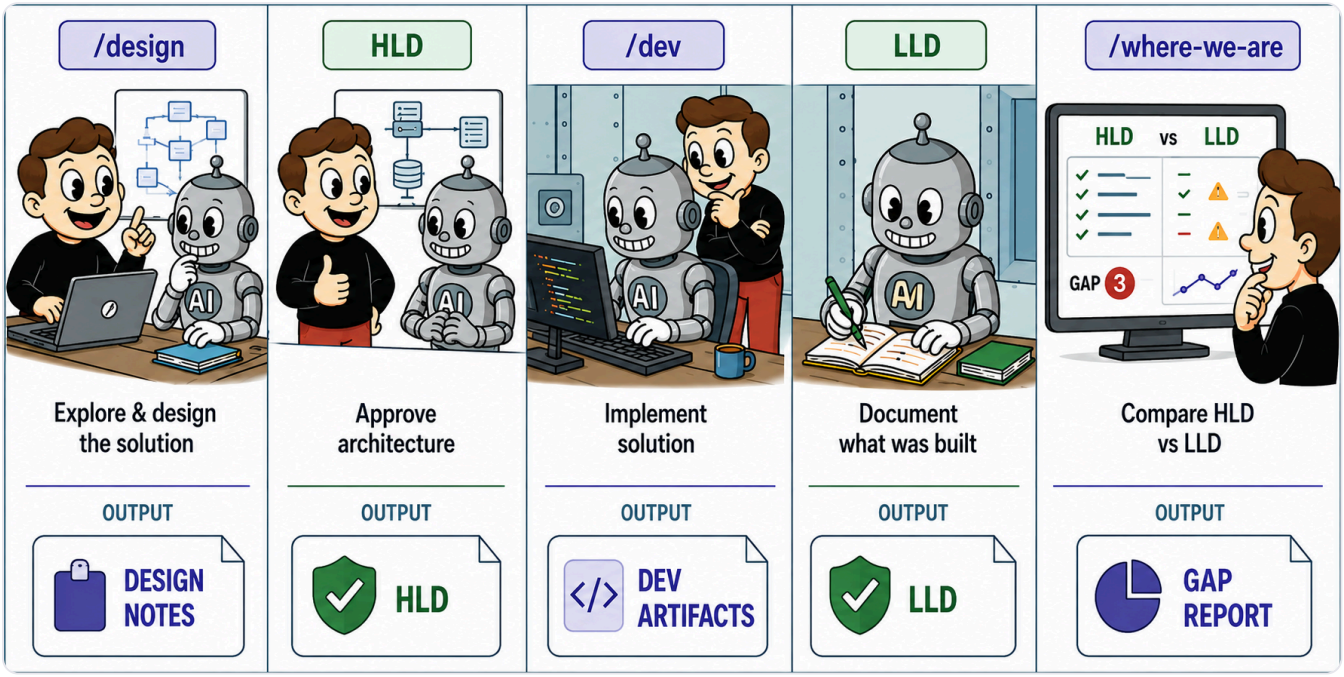
"Code is a derivative. Documents are the product." A decision doesn't start as code and get documented after; it starts as a settled decision and code follows from it. Frame states the problem → Elaborate produces a falsifiable recommendation → Solution writes `Architecture.md` (still no code) → Publish updates the HLD with that settled decision → `/dev` implements against the HLD → `/dev-document` checks what got built against it and records any deviation explicitly, rather than letting the code silently become the only record of what was decided.

PART THREE — HOW IT WORKS

04 How it works

"Code is a derivative. Documents are the product."

The same five checkpoints on every ticket, each one confirmed by a human before the next starts: a proposal, an approved architecture, an implementation, a record of what was built, and a report on whether the two still agree.



The five checkpoints in miniature. The five subsections that follow are the mechanics behind each one — plus Skill, the loop that improves all of them over time.

05 Design — from problem to approved change

Front-loads the hard thinking, and requires a human out of band before any code is written.

Frame
Problem, known + proposed requirements, boundaries

→

Research
`@architect` returns 3–5 options, no recommendation

→

Elaborate
Requirements × Options matrix, falsifiable recommendation

→

Solution
Architecture · Design · Stack · Efforts — settled only

Review
Draft MR opened, WAF gate recorded

→

Address Feedback
Only if concerns raised — reiterates, re-approves

→

Publish
`docs/Target Architecture (HLD)/` updated, tickets created

The subfolder is resolved once, in Frame, and every later phase reads it from `Report.md`'s `## Context` section rather than re-deriving it. A closing WAF assessment runs on the recommended option before Solution begins.

06 Dev — from approved change to merged code

The default path for well-understood work — no option analysis needed, just disciplined execution.

Plan
`/dev-plan` proposes an implementation plan for developer verification

→

Implement
`/dev-implement` executes it one step at a time

→

Test
`/dev-test` runs a layer-delegated PR review

→

Document
`/dev-document` updates READMEs, LLD, Engineering Guide

Every step stops and waits for developer confirmation before the next one starts — the plan is a proposal to verify, not a script to trust blindly. The story folder archives once Document completes.

07 Skill — where the harness improves itself

The mechanism that makes every future ticket faster than the last.

PHASE	WHAT HAPPENS
Reflect	Brain-dump what surprised you after a ticket — into `.claude/reflections/`
Triage	Cluster pending reflections into patterns worth capturing
New	Draft a skill or rule from a cluster — saved to `.claude/skills/drafts/`
Promote	Graduate a draft to active status, register it in `CLAUDE.md`
Harvest	Sync the promoted skill back into `docs/Harness/package/`, stripped to generic `{{PLACEHOLDER}}` form

THE CONVERGENCE LOOP

A project's live `.claude/` files are always the source of truth; `docs/Harness/package/` holds the generic, parametrized form. Harvest is the one-way door between them — nothing reaches the portable package without first proving itself on a real ticket in a real project.

08 Self-document — where work graduates

Documentation is not a separate chore — it's a required phase output. `/dev-document` and `/design-publish` are where a ticket or design graduates into permanent docs, and neither lifecycle can close without running it.

PRODUCES	LIVES IN
LLD updates	`System Architecture (LLD)/`
Engineering Guide entries	`Engineering Guide/`
App README updates	`apps/*/README.md`
HLD updates (Design track)	`Target Architecture (HLD)/<active>/`

Only settled outcomes graduate — rationale, comparisons, and open questions stay in the ticket's own `Stories/<TICKET>/` working folder and are never copied into permanent docs.

THE PACKAGE, IN PASSING

`docs/Harness/package/` — the copyable, self-contained form of everything in this handbook — is excluded from this site's own build (its `{{PLACEHOLDER}}` template syntax collides with Vue's `{{ }}` interpolation). Fully readable on GitHub or in a checkout.

09 Architect — the skill set behind it

The harness is not one general-purpose assistant — it's a roster of agents, each with a narrow remit and a specific tool grant, invoked by the lifecycle skill that needs them.

AGENT	REMIT	TOOLS
@architect	Review + author architecture docs; Design's Research/Review/Publish reviewer	Read, Glob, Grep, Bash, Write, Edit
@docs-writer	Permanent doc writer — diagram-first, certainty principle, destination rules	Read, Write, Edit, Glob, Grep
@<layer>	Generic layer template, instantiated once per bounded application layer at bootstrap — this repo's instance is `@web` / `@auth` / `@infra`, one per app	Read, Edit, Write, Glob, Grep
@onboarding	Conversational, read-only guide for anyone new to the repo	Read, Glob, Grep

SCOPE IS THE SAFETY MECHANISM

An agent restricted to `apps/lambda-auth/` cannot accidentally edit infrastructure. Combined with guarded commands — `cdk deploy`, `git push` are always presented as instructions for the developer to run, never executed by an agent directly — the tool grant is what keeps a lifecycle-driven session from acting outside its stated job.